CaveAgent

Transforming LLMs into Stateful Runtime Operators

Maohao Ran*, Zhenglin Wan*, Cooper Lin, Yanting Zhang, Hongyu Xin, et al.

HKUST · HKBU · NUS · HKU · CMU · NTU · Imperial · Princeton · UNC · Harvard

Hong Kong Generative AI Research and Development Center (HKGAI)

arXiv: 2601.01569
GitHub: github.com/caveagent/cave-agent · PyPI: cave-agent v0.6.5

关于我们

万政霖 Zhenglin (Carlos) Wan

Ph.D. Student, National University of Singapore (NUS)
HPC-AI Lab, Advisor: Prof. Yang You

研究方向

- LLM Agents with RL as core methodology
- RL post-training & world models in LLM Agents
- Self-evolving agents (Learning to Evolve)
- Agent systems: tool/skill/runtime/context management

代表性工作

- CaveAgent — 本次报告的工作， HKGAI 核心技术
- EBC: Diversifying Policy Behaviors (ICML 2025)
- POI Recommendation via Adversarial IL (AAAI 2025, oral)

vanzll.github.io

HKGAI

Hong Kong Generative AI
Research & Development Center

机构简介

- InnoHK 支持， 2023 年成立
- 香港最大的合作科研项目
- 由 HKUST 常务副校长 Prof. Yike Guo 领导
- 联合 6 所 QS 百强香港高校 + NUS

研究方向

- 多模态、多语言基础模型
- 垂直领域基础模型（法律、医疗、创意）
- HKGAI V1: 香港首个本地 AI 模型（粤语 / 普通话 / 英语）
- CaveAgent: 核心技术产品 HK E-Link

www.hkgai.info

背景与动机：JSON Function Calling 的局限

- LLM Agent 已无处不在，JSON function calling 是主流范式
- 但真实世界任务需要有状态的持久计算

```
// 典型 JSON function call
{
  "name": "get_weather",
  "arguments": {"location": "SF"}
}
→ Result returned as text string
→ No persistent state between calls
```

1. 序列化开销

复杂对象（DataFrame、模型）必须转为文本

2. 无持久状态

每次调用都是无状态的，变量无法跨步骤存活

3. 可组合性差

无法跨步骤对内存对象进行链式操作

结果：信息丢失 + 冗余计算 + context window 膨胀

文本化瓶颈（Textualization Bottleneck）

当前： Text-Bound Runtimes

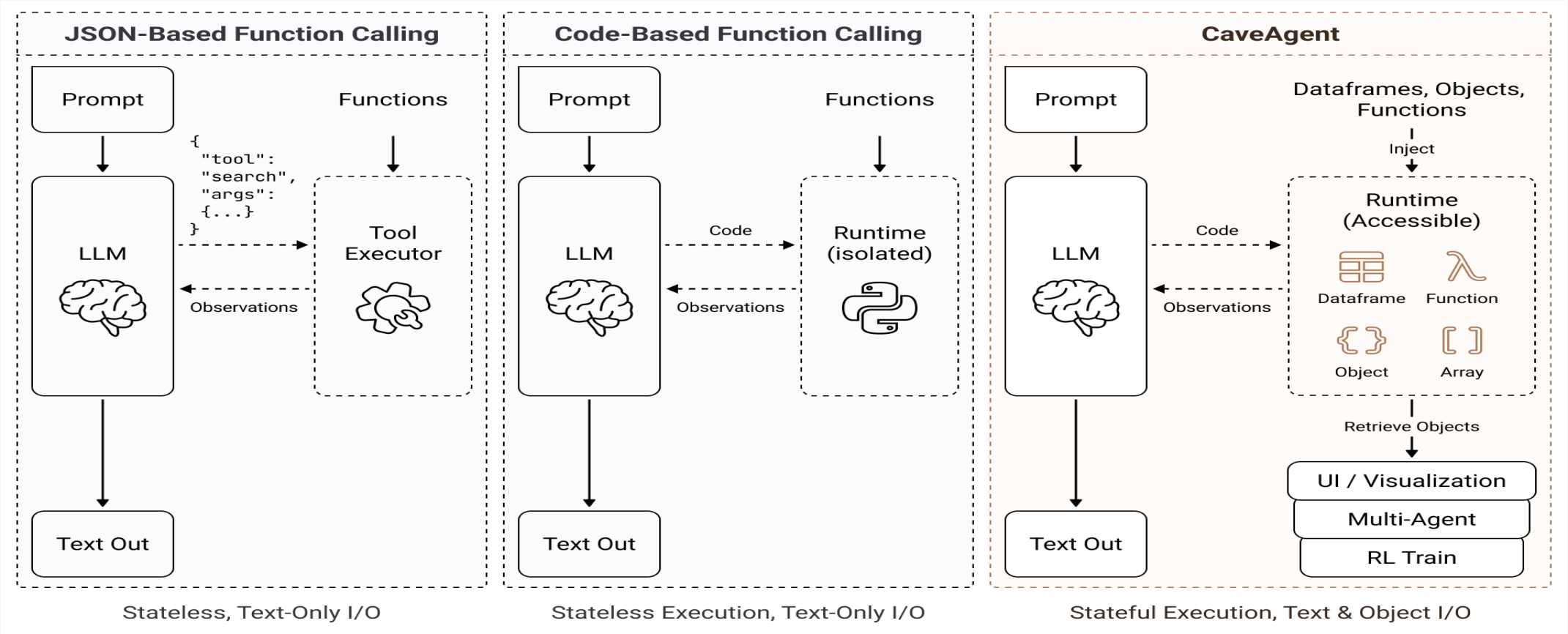
- 所有工具结果序列化为文本字符串
- DataFrame → 截断文本表格
- ML 模型 → 参数 dump
- DB 连接 → 每次重建

CaveAgent: Runtime-Native

- 对象直接活在持久 Python runtime 中
- DataFrame 直接操作，无需转换
- 模型加载一次，跨 turn 使用
- 只有摘要进入 context window

持久运行时消除了文本化瓶颈 — 对象始终是对象，而非字符串

LLM-Tool 交互范式的演进



Gen 0 Text-Only → Gen 1 JSON FC → Gen 2 Code-as-Action → Gen 3 Stateful Runtime (CaveAgent)

我们的愿景与贡献

“能否将 LLM 从文本生成器转变为有状态的运行时操作器，维护持久的计算环境？”

1

双流架构

Semantic Stream 用于推理 + Runtime Stream 用于持久执行，运行时技能管理

2

可编程验证

Runtime state 可确定性访问，支持自动化评测与 RLVR 奖励信号

3

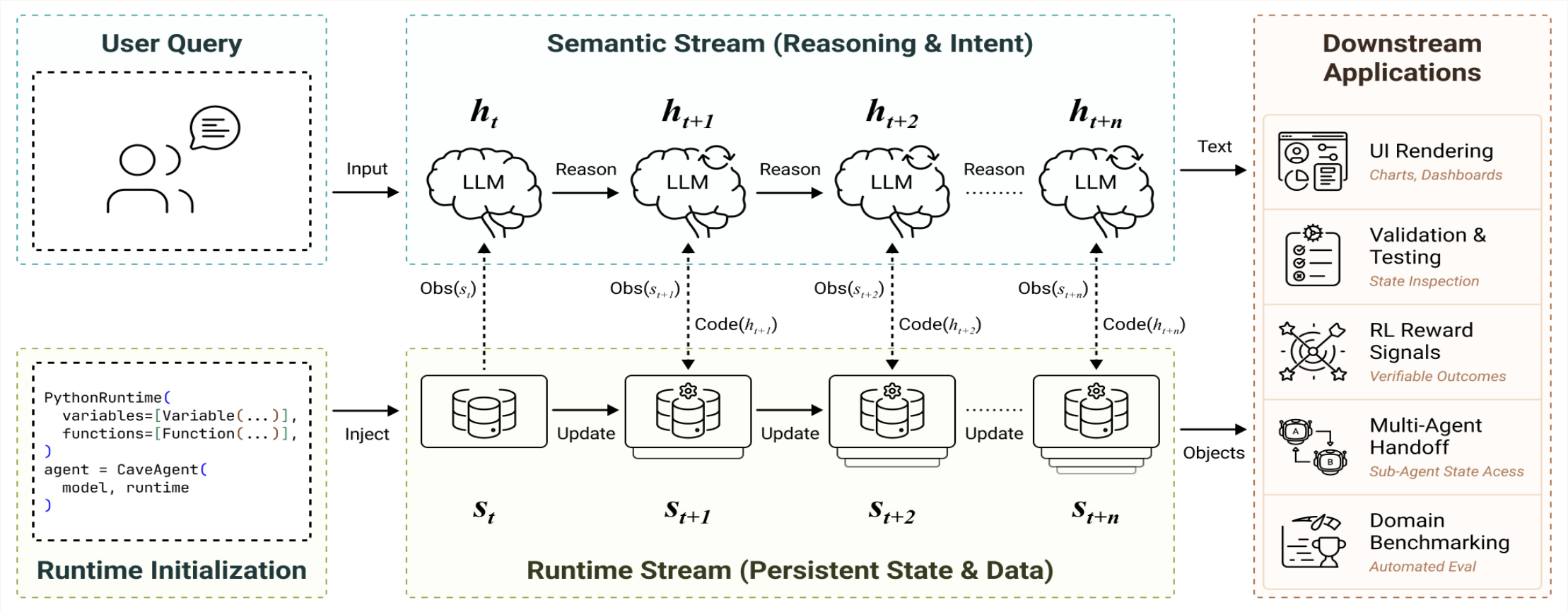
全面验证

6 个 SOTA LLM 、 Tau²-bench 、 BFCL 、自定义状态管理 benchmark

核心架构

CaveAgent Architecture

双流架构总览（Dual-Stream Architecture）



Semantic Stream （LLM 推理） + Runtime Stream （持久 Python 环境）

形式化模型与核心洞察

Semantic Stream

$$h_t = LLM(x_t, h_{t-1})$$

Context history — 会话 / 推理轨迹

Runtime Stream

$$S_t = Exec(c_t, S_{t-1})$$

持久环境 — 变量、对象、连接跨 turn 存活

核心洞察：解耦

- h_t = 语义历史（LLM 看到的）
- S_t = 运行时状态（持久存储的）
- N_t = 全局命名空间
- $\tau(\cdot)$ = 观察塑形函数

- Runtime 才是主要工作空间，
- 而不是 context window！

CaveAgent.run(): 构建提示 → 发送 LLM → 提取代码 → 执行 → 反馈 → 循环

运行时流与注入 API

```
from cave_agent import CaveAgent, Variable, Function

agent = CaveAgent(model=model)

# 注入真实 Python 对象
agent.runtime.inject(Variable(
    name="sales_data",
    value=df,  # 真实的 pandas DataFrame!
    description="Q1 2024 sales data"
))

# Step 1: df 持久存在
# Step 2: 直接聚合，无需重新加载
# Step 3: 在内存中构建模型
```

运行时原语

- Variable — 注入任意 Python 对象
- Function — 注入可调用函数
- Type — 注入自定义类型

- 基于 IPython 的持久内核
- 对象是真实的，不是序列化文本
- LLM 看到元数据摘要
- 代码操作真实对象

变量、对象、连接跨所有 agent step 持久存在 — 运行时是主要工作空间

上下文同步与安全模型

动态上下文同步

- Runtime State Summary: 系统提示自动包含当前变量 / 函数
- Observation Shaping (τ): 输出有 L_max 限制，大结果截断 + 摘要
- Progressive Disclosure: skill 元数据 ~100 tokens，按需加载

- 保持 context 紧凑，同时 runtime 无限扩展

AST-Based 安全模型

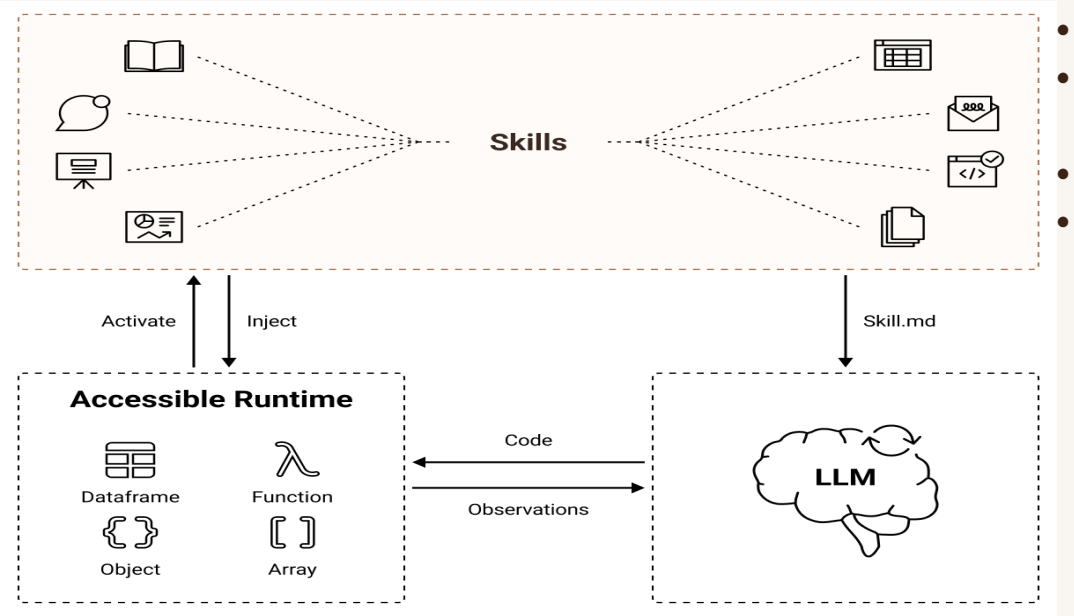
- ImportRule — 白名单 / 黑名单模块导入
- FunctionRule — 限制危险函数调用
- AttributeRule — 控制属性访问
- RegexRule — 模式扫描

- 自纠正流程：生成 → AST 检查 → 违规反馈 → 重新生成
- Pre-execution: 危险代码永不到达 runtime

技能与能力

Skills & Key Capabilities

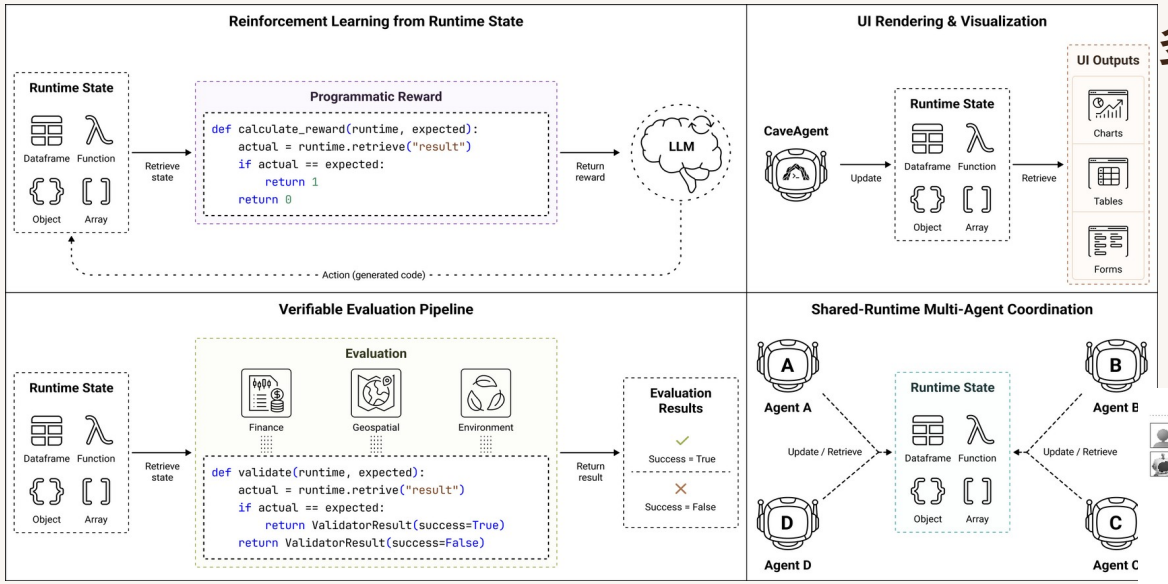
Runtime-Integrated Skills 架构



- 兼容 agentskills.io 开放标准
- 扩展：injection.py 中 __exports__ 导出 Function/Variable/Type
- 激活时直接注入 runtime，不只是文本指令
- 渐进披露：启动 ~100 tokens，按需加载完整指令

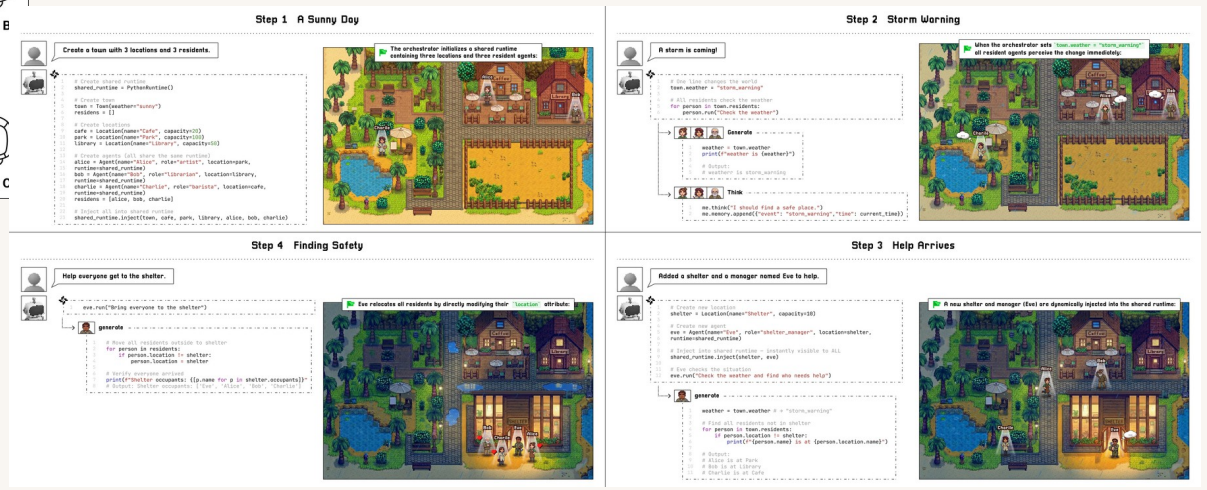
特性	Standard Skills	CaveAgent Skills
函数访问	文本描述， LLM 自行实现	可调用对象直接注入
状态管理	无 — 无状态	变量 / 类型持久在 runtime
可组合性	文本级链接	Python 代码组合对象
激活成本	完整 skill 文本加载	~100 tokens + 按需加载

下游应用与多 Agent 协作



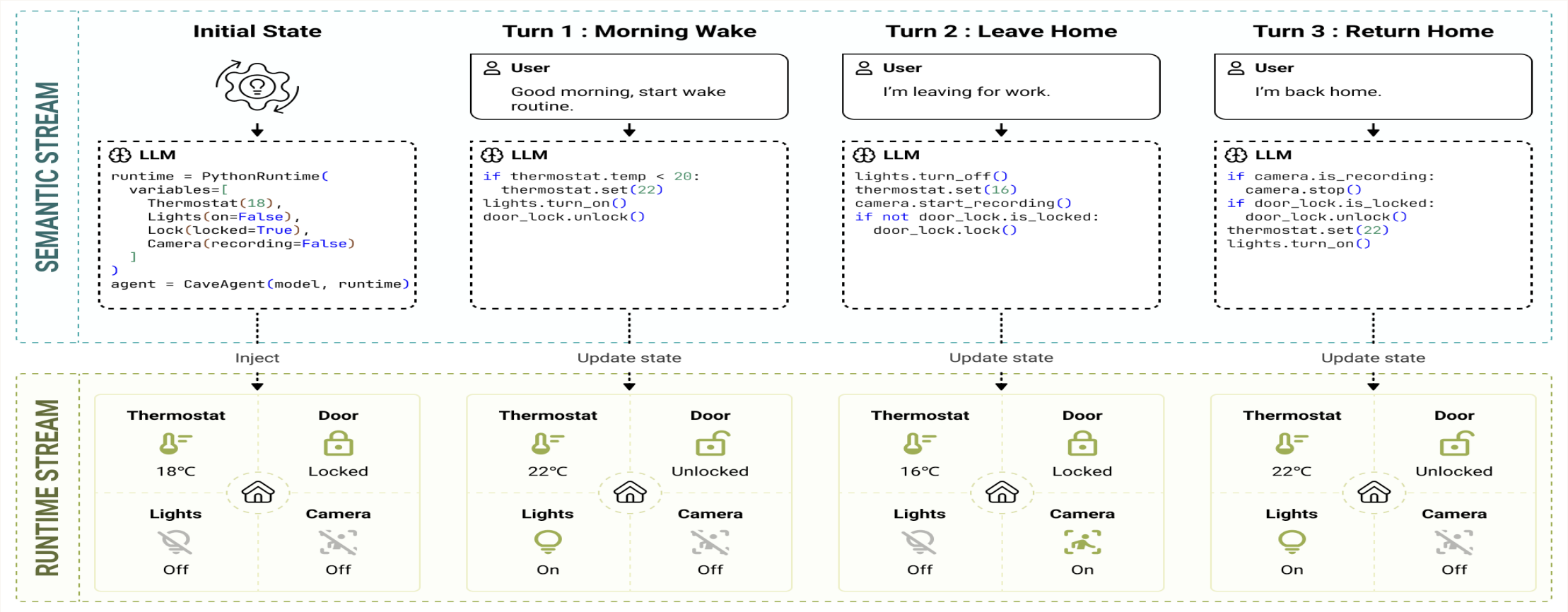
多 Agent 协作：共享 Runtime

- Agent 作为 runtime 中的对象
- 直接传递对象，无序列化开销
- Town simulation：多 Agent 交互演示



RLVR 基础： Runtime state 可确定性访问 → 自动化评测 + 细粒度奖励信号，无需人工标注

实际应用：智能家居控制



Agent 维护设备状态、场景、自动化规则，跨多轮交互持久保存

实验验证

Experiments & Results

实验设置

Benchmarks

- Tau²-bench (Airline + Retail)
- BFCL (Berkeley Function Calling Leaderboard)
- 自定义状态管理 Benchmark
- 数据密集任务 (query, analysis, visualization)

6 个 SOTA LLM

- 开源：DeepSeek-V3.2 (685B), Qwen3-Coder (30B), Kimi-K2 (1000B)
- 闭源：Claude Sonnet 4.5, GPT-5.1, Gemini 3 Pro
- 每个设置 3 次运行，保证统计可靠性

对比方式：同一模型、同一工具、同一任务 — 只换交互范式（JSON FC vs CaveAgent）

11/12
Tau²-bench 胜出

+40.9%
BFCL 最大提升

-28.4%
Token 减少

Tau²-bench 结果： 11/12 设置 CaveAgent 胜出

Model	Domain	FC	CaveAgent	Δ
DeepSeek-V3.2	Airline	55.3%	**60.0%**	**+4.7%**
	Retail	77.2%	**81.9%**	**+4.7%**
Qwen3-Coder	Airline	38.0%	**40.7%**	**+2.7%**
	Retail	41.2%	**54.7%**	**+13.5%**
Kimi-K2	Airline	54.0%	**55.3%**	**+1.3%**
	Retail	60.8%	**71.3%**	**+10.5%**
Claude 4.5	Airline	57.3%	56.7%	-0.6%
	Retail	72.5%	**76.6%**	**+4.1%**
GPT-5.1	Airline	52.7%	**56.0%**	**+3.3%**
	Retail	65.8%	**69.6%**	**+3.8%**
Gemini 3 Pro	Airline	61.3%	**68.0%**	**+6.7%**
	Retail	70.8%	**76.3%**	**+5.5%**

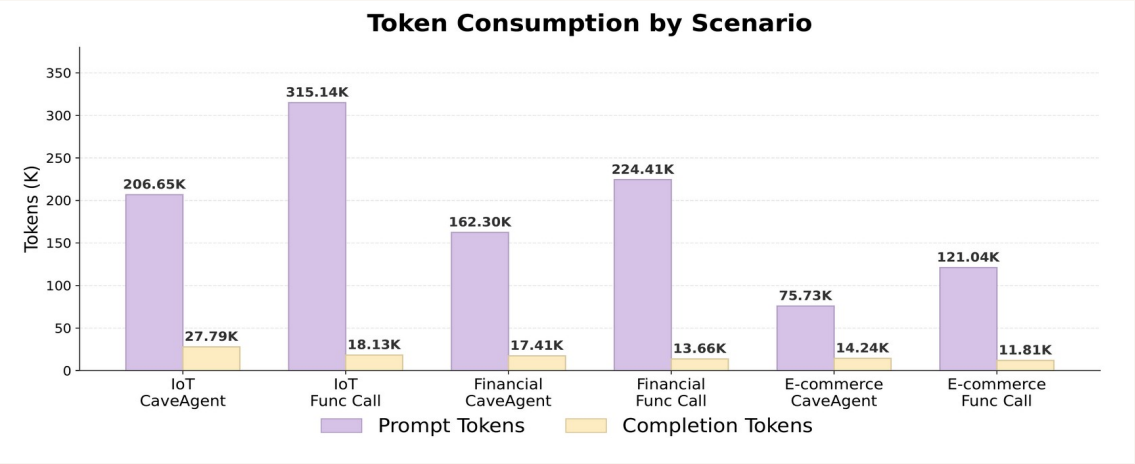
最大提升：+13.5% (Qwen3-Coder Retail) | 平均 ~4.7% | 唯一下降：Claude Airline -0.6%

BFCL 结果与 Token 效率

Model	FC	CaveAgent	Δ
DeepSeek-V3.2	86.9%	**94.0%**	**+7.1%**
DeepSeek (w/o prompt)	53.1%	**94.0%**	**+40.9%**
Qwen3-Coder	89.8%	**94.4%**	**+4.6%**
Kimi-K2	89.2%	**94.7%**	**+5.5%**
Claude 4.5	94.4%	**94.4%**	0.0%
GPT-5.1	89.6%	88.9%	-0.7%
Gemini 3 Pro	94.3%	**94.3%**	0.0%

-28.4%
Total Token 减少

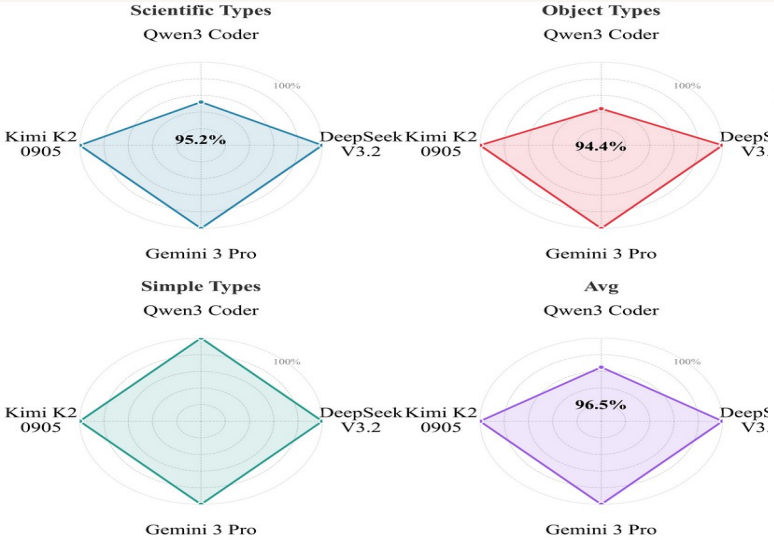
-38.6%
步骤减少



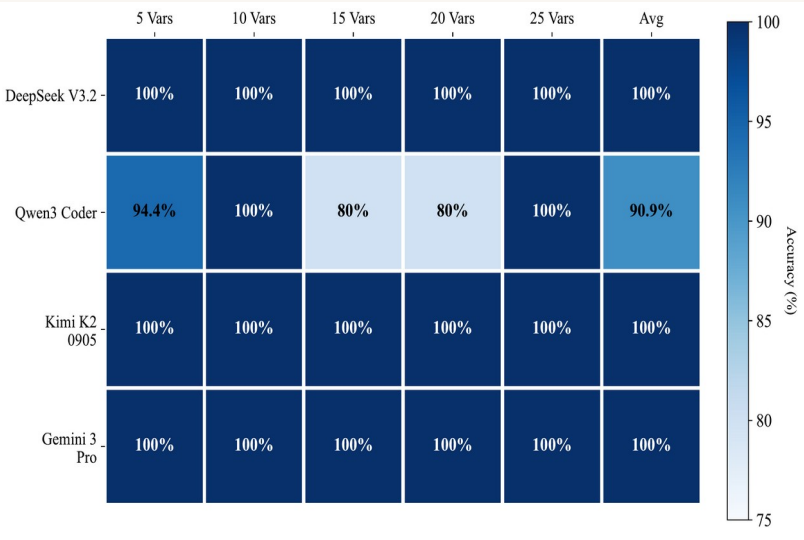
DeepSeek V3.2: 504K vs 704K tokens | 145 vs 236 steps | 100% vs 94.6% success

BFCL: DeepSeek 无 prompt 时 53.1% → 94.0% (+40.9%) — CaveAgent 对 prompt 格式的鲁棒性更强

状态管理 Benchmark 与案例研究



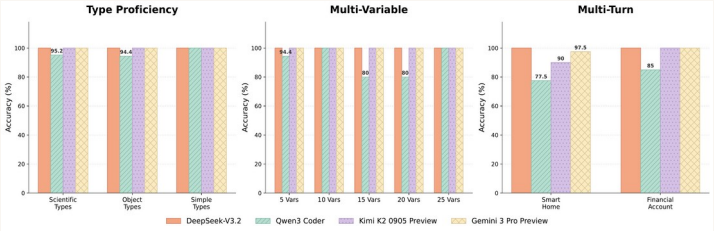
Type Proficiency

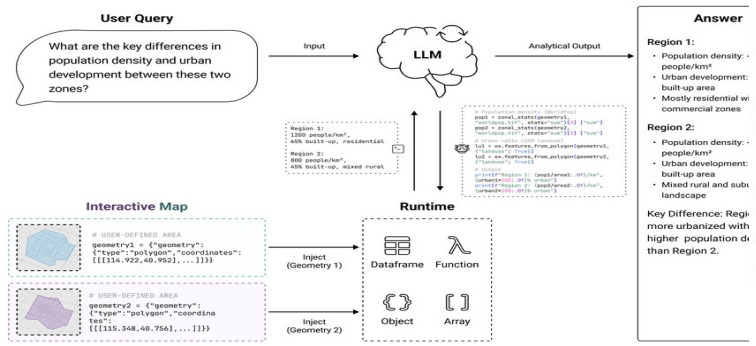


Multi-Variable (5V–25V)

任务	CaveAgent	CodeAct	JSON FC
Query	**100%**	80%	80%
Analysis	**100%**	100%	10%
Viz	**90%**	40%	30%
Viz tokens	**405K**	1M	662K

数据密集任务对比





Geospatial

AutoML Multi-Agent

总结

Conclusion

定位与贡献总结

Gen 0: Text-Only → Gen 1: JSON FC → Gen 2: Code-as-Action → Gen 3: Stateful Runtime

1

双流架构 + Skills

Semantic + Runtime stream
运行时技能管理

2

可编程验证 + RLVR

确定性状态检查
自动化奖励信号

3

全面实验验证

11/12 Tau²-bench
+40.9% BFCL, -28.4% tokens

从 text-bound 到 runtime-native 是根本性提升，显著改善了准确率、效率和状态管理能力

局限与未来方向

当前局限

- Python-centric：工具必须是 Python 可调用的
- 内存随存储对象复杂度增长
- 状态管理 benchmark 仍在早期阶段
- 多 Agent 协作仅有定性结果

未来方向

- 多 Agent 系统的严格定量评测
- 完善状态管理的综合 benchmark
- 长期任务的内存管理策略
- 跨语言工具生态集成

CaveAgent

谢谢！

问答与讨论

arXiv: 2601.01569
GitHub: github.com/caveagent/cave-agent
PyPI: pip install cave-agent

Contact: junsong@hkbu.edu.hk | vanzl@u.nus.edu